SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : S.RESHMITHA

Reg. No : 11249a339

Class : II B.E. (CSE)

Course Code:

Course Name: DBMS

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by **Mr/Ms.** S.RESHMITHA.

with **Reg. No** 11249a339 of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Station:

Date:

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

SL.NO	Program Name	Date	Page no	Signature
01	CREATE A DATABASE FOR SQL	09/08/26		
	QUERIES			
02	PERFORMING INSERTION,	09/08/26		
	DELETION, MODIFYING,			
	ALTERING AND VIEWS			
03	PERFORMING DCL AND TCL	15/08/26		
04	DATABASE OF VARIOUS	15/08/26		
	CONSTRAINTS			
05	PERFORMING QUERIES USING	24/08/26		
	ANY, ALL, IN, EXISTS, UNION			
06	AGGREGATE FUNCTIONS	24/08/26		
07	CREATE VIEWS, SEQUENCES	08/09/26		
	INDEXES, SAVEPOINTS			
08	NORMALIZATION IN ORACLE	08/09/26		
09	JOINS AND SUBQUERIES	16/09/26		

SL.NO	Program Name	Date	Page no	Signature
10	PL SQL ALL FUNCTIONS	16/09/26		
	PART A - F			
11	DATABASE FOR MANAGEMENT			
	SYSTEMS			

Experiment 1: Creation of a Database and Retrieval of Information

Aim: To create a database, create a table, insert records, and retrieve information using SQL queries.

Algorithm:

1. Create a database.
2. Select the database.
3. Create a table with suitable columns.
4. Insert records into the table.
5. Retrieve all records using the `SELECT` statement.
6. Retrieve specific records using the `WHERE` clause.

```
SQL> CREATE TABLE Crestview_University (
  2      StudentID INT,
  3      Name VARCHAR(30),
  4      Department VARCHAR(30),
  5      Year INT
  6  );

Table created.

SQL> INSERT INTO Crestview_University
  2  VALUES (101, 'Arun', 'CSE', 2);

1 row created.

SQL>
SQL> INSERT INTO Crestview_University
  2  VALUES (102, 'Priya', 'ECE', 3);

1 row created.

SQL>
SQL> INSERT INTO Crestview_University
  2  VALUES (103, 'Rahul', 'CSE', 1);

1 row created.

SQL> SELECT * FROM Crestview_University;

 STUDENTID NAME                           DEPARTMENT
---------- ------------------------------ ------------------------------
      YEAR
----------
       101 Arun                           CSE
         2
```

```
 row created.

QL> SELECT * FROM Crestview_University;

STUDENTID NAME                           DEPARTMENT
--------- ------------------------------ ----------
     YEAR
---------
      101 Arun                           CSE
        2

      102 Priya                          ECE
        3

      103 Rahul                          CSE
        1
```

```
QL> SELECT Name FROM Crestview_University;

AME
------------------------------
run
riya
ahul
```

EXPERIMENT 2

Result: The database and table were created successfully, and the required information was retrieved using SQL queries.

Experiment 2: Insertion, Deletion, Modification, Alteration and Updating of Records

Aim: To perform insertion, deletion, modification, alteration, and retrieval operations on a database table.

Algorithm:

1. Create a table and insert sample records.
2. Insert a new record using `INSERT`.
3. Modify a record using `UPDATE`.
4. Delete a record using `DELETE`.
5. Modify the table structure using `ALTER TABLE`.
6. Retrieve the final records using `SELECT`

```
SQL> CREATE TABLE Aurora_Sky_Tower (
  2      TowerID NUMBER,
  3      Name VARCHAR2(30),
  4      Floor NUMBER,
  5      Rent NUMBER
  6  );

Table created.

SQL> INSERT INTO Aurora_Sky_Tower
  2  VALUES (101, 'Office A', 5, 25000);

1 row created.

SQL>
SQL> INSERT INTO Aurora_Sky_Tower
  2  VALUES (102, 'Office B', 8, 35000);

1 row created.

SQL>
SQL> INSERT INTO Aurora_Sky_Tower
  2  VALUES (103, 'Office C', 3, 20000);

1 row created.

SQL> SELECT * FROM Aurora_Sky_Tower;

   TOWERID NAME                                FLOOR       RENT
---------- ------------------------------ ---------- ----------
       101 Office A                                5      25000
       102 Office B                                8      35000
       103 Office C                                3      20000

SQL> UPDATE Aurora_Sky_Tower
  2  SET Rent = 28000
  3  WHERE TowerID = 101;

1 row updated.

SQL> DELETE FROM Aurora_Sky_Tower
  2  WHERE TowerID = 103;
```

```
1 row updated.

SQL> DELETE FROM Aurora_Sky_Tower
  2  WHERE TowerID = 103;

1 row deleted.

SQL> ALTER TABLE Aurora_Sky_Tower
  2  MODIFY Name VARCHAR2(50);

Table altered.

SQL> SELECT * FROM Aurora_Sky_Tower
  2  WHERE Rent > 25000;
```

OUTPUT

```
 TOWERID NAME                                               FLOOR
-------- -------------------------------------------------- ----------
    RENT
---------
     101 Office A                                                5
   28000

     102 Office B                                                8
   35000
```

Result: Insertion, deletion, updating, alteration, and retrieval operations were performed successfully.

Experiment 3: Practicing DDL, DML and TCL Commands

Aim: To practice Data Definition Language (DDL), Data Manipulation Language (DML), and Transaction Control Language (TCL) commands.

Algorithm:

1. Create a table using DDL commands.
2. Insert and modify records using DML commands.
3. Start a transaction where supported.
4. Use `SAVEPOINT` to mark a transaction point.
5. Use `ROLLBACK` to undo selected changes.
6. Use `COMMIT` to save the remaining changes.

```
SQL> CREATE TABLE Nomad_Quarters (
  2      RoomID NUMBER,
  3      GuestName VARCHAR2(30),
  4      RoomType VARCHAR2(20),
  5      Rent NUMBER
  6  );

Table created.

SQL> INSERT INTO Nomad_Quarters
  2  VALUES (101, 'Arun', 'Single', 2000);

1 row created.

SQL>
SQL> INSERT INTO Nomad_Quarters
  2  VALUES (102, 'Priya', 'Double', 3000);

1 row created.

SQL>
SQL> INSERT INTO Nomad_Quarters
  2  VALUES (103, 'Rahul', 'Single', 2200);

1 row created.

SQL> GRANT SELECT ON Nomad_Quarters TO user1;

Grant succeeded.

SQL> REVOKE SELECT ON Nomad_Quarters FROM user1;

Revoke succeeded.

SQL> COMMIT;

Commit complete.

SQL> UPDATE Nomad_Quarters
  2  SET Rent = 2500
  3  WHERE RoomID = 101;

1 row updated
```

```
Commit complete.

SQL> UPDATE Nomad_Quarters
  2  SET Rent = 2500
  3  WHERE RoomID = 101;

1 row updated.

SQL> ROLLBACK;

Rollback complete.

SQL> SAVEPOINT S1;

Savepoint created.

SQL> ROLLBACK TO S1;

Rollback complete.

SQL> SELECT * FROM Nomad_Quarters;
```

OUTPUT

```
    ROOMID GUESTNAME                      ROOMTYPE                 RENT
---------- ------------------------------ -------------------- ----------
       101 Arun                           Single                     2000
       102 Priya                          Double                     3000
       103 Rahul                          Single                     2200

SQL> _
```

Result: DDL, DML, and TCL commands were executed to demonstrate database definition, manipulation, and transaction control.

Experiment 4: Creating a Database Table with Constraints

Aim: To create a table and apply different constraints such as `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, `CHECK`, and `DEFAULT`.

Algorithm:

1Create a parent table with a primary key.

2.Create a child table with a foreign key.

3.Apply the required constraints.

4.Insert valid records.

5.Retrieve the records to verify the constraints

```
SQL> CREATE TABLE Grandview_Plaza (
  2      ShopID NUMBER PRIMARY KEY,
  3      ShopName VARCHAR2(30) NOT NULL,
  4      OwnerName VARCHAR2(30),
  5      Phone VARCHAR2(15) UNIQUE,
  6      Floor NUMBER CHECK (Floor >= 1),
  7      ShopType VARCHAR2(20) DEFAULT 'General',
  8      Rent NUMBER CHECK (Rent > 0)
  9  );

Table created.

SQL> INSERT INTO Grandview_Plaza
  2  VALUES (101, 'Trends', 'Arun', '9876543210', 1, 'Fashion', 25000);

1 row created.

SQL>
SQL> INSERT INTO Grandview_Plaza
  2  VALUES (102, 'Fresh Mart', 'Priya', '9876543211', 2, 'Grocery', 30000);

1 row created.

SQL>
SQL> INSERT INTO Grandview_Plaza
  2  VALUES (103, 'Tech World', 'Rahul', '9876543212', 3, 'Electronics', 40000);

1 row created.

SQL>
SQL> INSERT INTO Grandview_Plaza
  2  VALUES (104, 'Cafe Bliss', 'Meena', '9876543213', 1, 'Food', 20000);

1 row created.

SQL> INSERT INTO Grandview_Plaza
  2  VALUES (101, 'Max Store', 'Kumar', '9876543215', 2, 'Fashion', 22000);
INSERT INTO Grandview_Plaza
*
ERROR at line 1:
ORA-00001: unique constraint (A339.SYS_C0033607) violated
```

```
876543212                3 Electronics                40000

   SHOPID SHOPNAME                       OWNERNAME
--------- ------------------------------ ------------------------
PHONE                FLOOR SHOPTYPE                    RENT
-------------- ---------- -------------------- ----------
      104 Cafe Bliss                        Meena
876543213                1 Food                       20000

      105 Book Store                        Ravi
876543216                2 General                    18000

QL> SELECT * FROM Grandview_Plaza;

   SHOPID SHOPNAME                       OWNERNAME
--------- ------------------------------ ------------------------
PHONE                FLOOR SHOPTYPE                    RENT
-------------- ---------- -------------------- ----------
      101 Trends                            Arun
876543210                1 Fashion                    25000

      102 Fresh Mart                        Priya
876543211                2 Grocery                    30000

      103 Tech World                        Rahul
876543212                3 Electronics                40000

   SHOPID SHOPNAME                       OWNERNAME
--------- ------------------------------ ------------------------
PHONE                FLOOR SHOPTYPE                    RENT
-------------- ---------- -------------------- ----------
      104 Cafe Bliss                        Meena
876543213                1 Food                       20000

      105 Book Store                        Ravi
876543216                2 General                    18000

QL> SELECT * FROM Grandview_Plaza
 2  WHERE Floor = 1;^B
```

Result: The tables were created successfully with the required integrity constraints.

Experiment 5: Queries Using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION and INTERSECT

Aim: To execute SQL queries using comparison operators, subqueries, and set operators.

Algorithm:

1. Create and populate the required tables.
2. Use `IN` to match values in a subquery.
3. Use `ANY` and `ALL` for comparisons against multiple values.
4. Use `EXISTS` and `NOT EXISTS` to test for matching records.
5. Use `UNION` and `INTERSECT` to combine query results.

```
SQL> CREATE TABLE Grandview_Plaza (
  2      ShopID NUMBER PRIMARY KEY,
  3      ShopName VARCHAR2(30) NOT NULL,
  4      OwnerName VARCHAR2(30),
  5      Phone VARCHAR2(15) UNIQUE,
  6      Floor NUMBER CHECK (Floor >= 1),
  7      ShopType VARCHAR2(20) DEFAULT 'General',
  8      Rent NUMBER CHECK (Rent > 0)
  9  );

Table created.

SQL> INSERT INTO Grandview_Plaza
  2  VALUES (101, 'Trends', 'Arun', '9876543210', 1, 'Fashion', 25000);

1 row created.

SQL>
SQL> INSERT INTO Grandview_Plaza
  2  VALUES (102, 'Fresh Mart', 'Priya', '9876543211', 2, 'Grocery', 30000);

1 row created.

SQL>
SQL> INSERT INTO Grandview_Plaza
  2  VALUES (103, 'Tech World', 'Rahul', '9876543212', 3, 'Electronics', 40000);

1 row created.

SQL>
SQL> INSERT INTO Grandview_Plaza
  2  VALUES (104, 'Cafe Bliss', 'Meena', '9876543213', 1, 'Food', 20000);

1 row created.

SQL> INSERT INTO Grandview_Plaza
  2  VALUES (101, 'Max Store', 'Kumar', '9876543215', 2, 'Fashion', 22000);
INSERT INTO Grandview_Plaza
*
ERROR at line 1:
ORA-00001: unique constraint (A339.SYS_C0033607) violated
```

```
QL>
QL> INSERT INTO Willow_Creek_Room VALUES
 2  (102, 'Double', 'Bangalore', 22000);

 row created.

QL>
QL> INSERT INTO Willow_Creek_Room VALUES
 2  (103, 'Suite', 'Hyderabad', 30000);

 row created.

QL>
QL> INSERT INTO Willow_Creek_Room VALUES
 2  (104, 'Single', 'Mumbai', 18000);

 row created.

QL> INSERT INTO Willow_Creek_Guest VALUES
 2  (1, 'Arun', 'Chennai', 101);

 row created.

QL>
QL> INSERT INTO Willow_Creek_Guest VALUES
 2  (2, 'Priya', 'Bangalore', 102);

 row created.

QL>
QL> INSERT INTO Willow_Creek_Guest VALUES
 2  (3, 'Rahul', 'Hyderabad', 103);

 row created.

QL> SELECT *
 2  FROM Willow_Creek_Room
 3  WHERE Location IN ('Chennai', 'Mumbai');

   ROOMID ROOMTYPE                       LOCATION
--------- ------------------------------ ------------------------------
     RENT
```

```
 row created.

QL> SELECT *
  2  FROM Willow_Creek_Room
  3  WHERE Location IN ('Chennai', 'Mumbai');

   ROOMID ROOMTYPE                       LOCATION
--------- ----------------------------- -------------------------
     RENT
---------
      101 Single                        Chennai
    15000

      104 Single                        Mumbai
    18000


QL> SELECT *
  2  FROM Willow_Creek_Room
  3  WHERE Rent > ANY
  4  (
  5      SELECT Rent
  6      FROM Willow_Creek_Room
  7      WHERE Location = 'Chennai'
  8  );

   ROOMID ROOMTYPE                       LOCATION
--------- ----------------------------- -------------------------
     RENT
---------
      103 Suite                         Hyderabad
    30000

      102 Double                        Bangalore
    22000

      104 Single                        Mumbai
    18000


QL> SELECT *
  2  FROM Willow_Creek_Room
  3  WHERE Rent > ALL
```

```
SQL> SELECT *
  2  FROM Willow_Creek_Room
  3  WHERE Rent > ALL
  4  (
  5      SELECT Rent
  6      FROM Willow_Creek_Room
  7      WHERE Location IN ('Chennai', 'Mumbai')
  8  );

    ROOMID ROOMTYPE                       LOCATION
---------- ------------------------------ ------------------------------
      RENT
----------
       102 Double                         Bangalore
     22000

       103 Suite                          Hyderabad
     30000


SQL> SELECT *
  2  FROM Willow_Creek_Room R
  3  WHERE EXISTS
  4  (
  5      SELECT *
  6      FROM Willow_Creek_Guest G
  7      WHERE G.RoomID = R.RoomID
  8  );

    ROOMID ROOMTYPE                       LOCATION
---------- ------------------------------ ------------------------------
      RENT
----------
       101 Single                         Chennai
     15000

       102 Double                         Bangalore
     22000

       103 Suite                          Hyderabad
     30000
```

```
SQL> SELECT *
  2  FROM Willow_Creek_Room R
  3  WHERE EXISTS
  4  (
  5      SELECT *
  6      FROM Willow_Creek_Guest G
  7      WHERE G.RoomID = R.RoomID
  8  );

    ROOMID ROOMTYPE                       LOCATION
---------- ------------------------------ ------------------------------
      RENT
----------
       101 Single                         Chennai
     15000

       102 Double                         Bangalore
     22000

       103 Suite                          Hyderabad
     30000


SQL> SELECT *
  2  FROM Willow_Creek_Room R
  3  WHERE NOT EXISTS
  4  (
  5      SELECT *
  6      FROM Willow_Creek_Guest G
  7      WHERE G.RoomID = R.RoomID
  8  );

    ROOMID ROOMTYPE                       LOCATION
---------- ------------------------------ ------------------------------
      RENT
----------
       104 Single                         Mumbai
     18000


SQL> SELECT Location
  2  FROM Willow_Creek_Room
```

```
QL> SELECT * FROM Willow_Creek_Guest;

  GUESTID GUESTNAME                      LOCATION
--------- ------------------------------ ------------------------------
   ROOMID
---------
        1 Arun                           Chennai
      101

        2 Priya                          Bangalore
      102

        3 Rahul                          Hyderabad
      103
```

Result: The tables were created successfully with the required integrity constraints.

Experiment 5: Queries Using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION and INTERSECT

Aim: To execute SQL queries using comparison operators, subqueries, and set operators.

Algorithm:

1. Create and populate the required tables.
2. Use `IN` to match values in a subquery.
3. Use `ANY` and `ALL` for comparisons against multiple values.
4. Use `EXISTS` and `NOT EXISTS` to test for matching records.
5. Use `UNION` and `INTERSECT` to combine query results.

```
SQL> CREATE TABLE Crestview_University (
  2      StudentID INT,
  3      Name VARCHAR(30),
  4      Department VARCHAR(30),
  5      Year INT
  6  );

Table created.

SQL> INSERT INTO Crestview_University
  2  VALUES (101, 'Arun', 'CSE', 2);

1 row created.

SQL>
SQL> INSERT INTO Crestview_University
  2  VALUES (102, 'Priya', 'ECE', 3);

1 row created.

SQL>
SQL> INSERT INTO Crestview_University
  2  VALUES (103, 'Rahul', 'CSE', 1);

1 row created.

SQL> SELECT * FROM Crestview_University;

 STUDENTID NAME                           DEPARTMENT
---------- ------------------------------ ------------------------------
      YEAR
----------
       101 Arun                           CSE
         2

       102 Priya                          ECE
```

```
SQL> SELECT *
  2  FROM Willow_Creek_Room
  3  WHERE Rent > ALL
  4  (
  5      SELECT Rent
  6      FROM Willow_Creek_Room
  7      WHERE Location IN ('Chennai', 'Mumbai')
  8  );

   ROOMID ROOMTYPE                       LOCATION
--------- ------------------------------ ------------------------------
     RENT
----------
      102 Double                         Bangalore
    22000

      103 Suite                          Hyderabad
    30000


SQL> SELECT *
  2  FROM Willow_Creek_Room R
  3  WHERE EXISTS
  4  (
  5      SELECT *
  6      FROM Willow_Creek_Guest G
  7      WHERE G.RoomID = R.RoomID
  8  );

   ROOMID ROOMTYPE                       LOCATION
--------- ------------------------------ ------------------------------
     RENT
----------
      101 Single                         Chennai
    15000

      102 Double                         Bangalore
    22000

      103 Suite                          Hyderabad
```

```
L> SELECT * FROM Willow_Creek_Guest;

 GUESTID GUESTNAME                      LOCATION
-------- ------------------------------ ------------------------------
  ROOMID
--------
       1 Arun                           Chennai
     101

       2 Priya                          Bangalore
     102

       3 Rahul                          Hyderabad
     103
```

Experiment 6: Aggregate Functions, GROUP BY and HAVING

Aim: To perform calculations using aggregate functions and group records based on specified conditions.

Algorithm:

1. Create a student marks table.
2. Insert sample records.
3. Apply aggregate functions such as `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX`.
4. Group records using `GROUP BY`.
5. Filter grouped results using `HAVING`.

```
SQL> CREATE TABLE Crestview_University (
  2      StudentID INT,
  3      Name VARCHAR(30),
  4      Department VARCHAR(30),
  5      Year INT
  6  );

Table created.

SQL> INSERT INTO Crestview_University
  2  VALUES (101, 'Arun', 'CSE', 2);

1 row created.

SQL>
SQL> INSERT INTO Crestview_University
  2  VALUES (102, 'Priya', 'ECE', 3);

1 row created.

SQL>
SQL> INSERT INTO Crestview_University
  2  VALUES (103, 'Rahul', 'CSE', 1);

1 row created.

SQL> SELECT * FROM Crestview_University;

 STUDENTID NAME                           DEPARTMENT
---------- ------------------------------ ------------------------------
      YEAR
----------
       101 Arun                           CSE
         2

       102 Priya                          ECE
         3

       103 Rahul                          CSE
         1
```

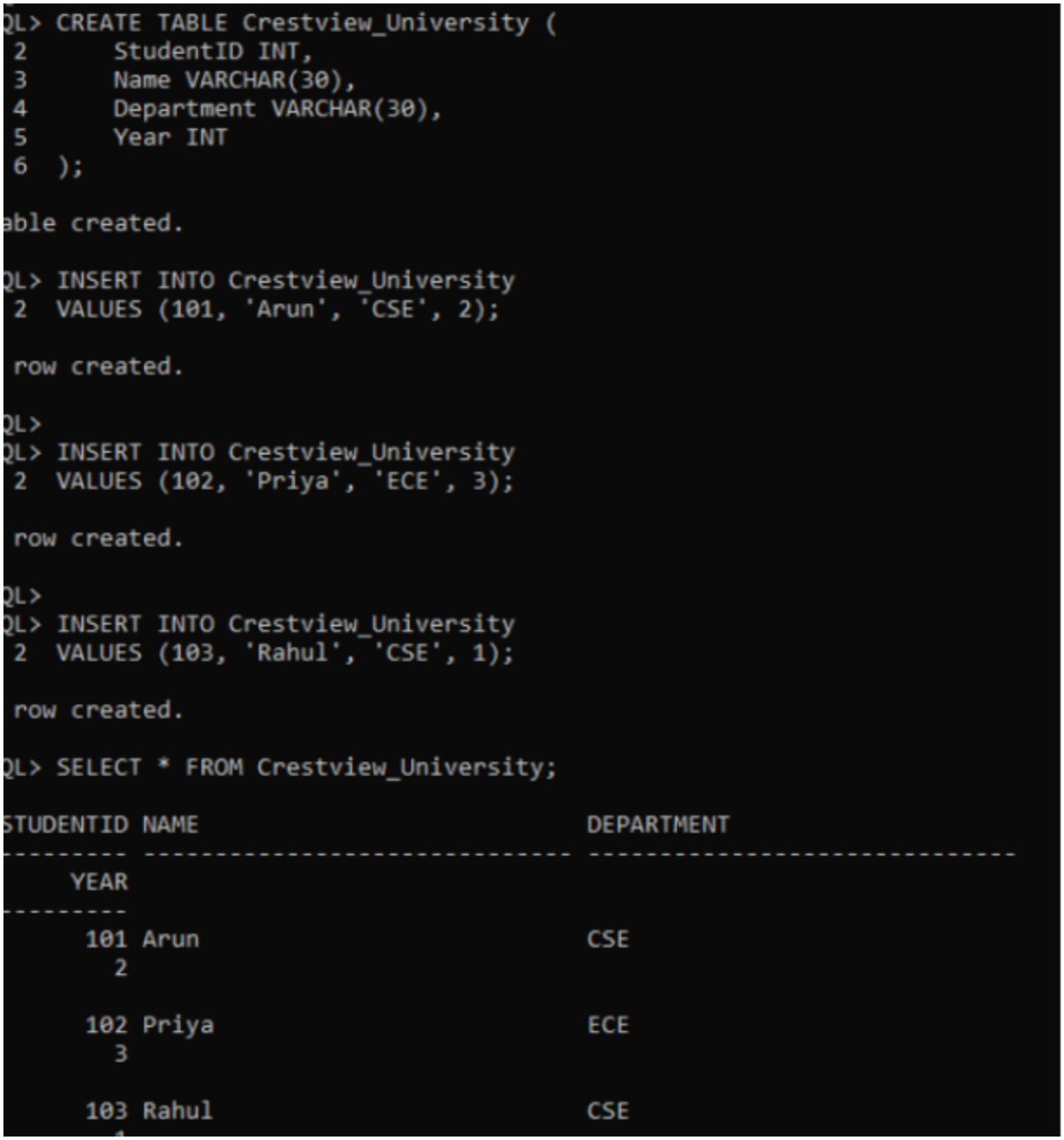
Result: Aggregate functions, grouping, and group filtering were executed successfully

Experiment 7: Views, Sequences, Indexes and Savepoints

Aim: To create and execute views, sequences, indexes, and savepoints using SQL commands.

Algorithm

1. Create an employee table.
2. Insert sample records into the table.
3. Create a view to display selected columns.
4. Create a sequence to generate unique numbers.
5. Create an index on the employee name.
6. Use a savepoint during a transaction.
7. Roll back to the savepoint and display the final records.



```
QL> CREATE TABLE Crestview_University (
 2      StudentID INT,
 3      Name VARCHAR(30),
 4      Department VARCHAR(30),
 5      Year INT
 6  );

able created.

QL> INSERT INTO Crestview_University
 2  VALUES (101, 'Arun', 'CSE', 2);

 row created.

QL>
QL> INSERT INTO Crestview_University
 2  VALUES (102, 'Priya', 'ECE', 3);

 row created.

QL>
QL> INSERT INTO Crestview_University
 2  VALUES (103, 'Rahul', 'CSE', 1);

 row created.

QL> SELECT * FROM Crestview_University;

STUDENTID NAME                           DEPARTMENT
--------- ------------------------------ ------------------------------
     YEAR
---------
      101 Arun                           CSE
        2

      102 Priya                          ECE
        3

      103 Rahul                          CSE
```

Result

Views, sequences, indexes, and savepoints were created and executed successfully.

Experiment 8: Normalization Using Functional Dependencies

Aim: To normalize a relation into First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), Fourth Normal Form (4NF), and Fifth Normal Form (5NF), as applicable.

Algorithm:

1. Identify the attributes and functional dependencies.
2. Identify the candidate key.
3. Convert the relation into 1NF by ensuring atomic values.
4. Convert into 2NF by removing partial dependencies.
5. Convert into 3NF by removing transitive dependencies.
6. Check for multivalued dependencies and join dependencies to determine whether 4NF and 5NF decompositions are necessary.

Consider the relation:

`STUDENT(RollNo, StudentName, DeptID, DeptName)`

Functional dependencies:

- `RollNo → StudentName, DeptID`
- `DeptID → DeptName`

Step 1: First Normal Form (1NF)

Ensure each cell contains only one value.

RollNo	StudentName	DeptID	DeptName
101	Anu	10	CSE
102	Ravi	20	ECE

Step 2: Second Normal Form (2NF)

The key is the single attribute `RollNo`, so there can be no partial dependency on part of a composite key. The relation already satisfies 2NF, assuming the values are atomic.

Step 3: Third Normal Form (3NF)

Since `DeptID → DeptName`, department name depends transitively on `RollNo`. Decompose the relation into two tables.

Code:

```
CREATE TABLE Department8 (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(30)
);

CREATE TABLE Student8 (
   RollNo INT PRIMARY KEY,
   StudentName VARCHAR(30),
   DeptID INT,
   FOREIGN KEY (DeptID)
      REFERENCES Department8(DeptID)
);

INSERT INTO Department8 VALUES
(10, 'CSE'),
(20, 'ECE');

INSERT INTO Student8 VALUES
(101, 'Anu', 10),
(102, 'Ravi', 20);

SELECT S.RollNo, S.StudentName,
      D.DeptID, D.DeptName
FROM Student8 S
JOIN Department8 D
ON S.DeptID = D.DeptID;
```

Output:

RollNo	StudentName	DeptID	DeptName
101	Anu	10	CSE
102	Ravi	20	ECE

Result: The functional dependencies were identified, and the relation was decomposed to remove the transitive dependency and achieve 3NF.

```
SQL> SELECT *
  2  FROM Willow_Creek_Room
  3  WHERE Rent > ALL
  4  (
  5      SELECT Rent
  6      FROM Willow_Creek_Room
  7      WHERE Location IN ('Chennai', 'Mumbai')
  8  );

    ROOMID ROOMTYPE                       LOCATION
---------- ------------------------------ ------------------------------
      RENT
----------
       102 Double                         Bangalore
     22000

       103 Suite                          Hyderabad
     30000


SQL> SELECT *
  2  FROM Willow_Creek_Room R
  3  WHERE EXISTS
  4  (
  5      SELECT *
  6      FROM Willow_Creek_Guest G
  7      WHERE G.RoomID = R.RoomID
  8  );

    ROOMID ROOMTYPE                       LOCATION
---------- ------------------------------ ------------------------------
      RENT
----------
       101 Single                         Chennai
     15000

       102 Double                         Bangalore
     22000

       103 Suite                          Hyderabad
     30000
```

Result: The functional dependencies were identified, and the relation was decomposed to remove the transitive dependency and achieve 3NF.

Experiment 9: Joins, Subqueries and PL/SQL Programming

Aim: To perform joins and subqueries and write PL/SQL programs using conditions, exception handling, procedures, functions, triggers, and cursors.

Part A: Joins and Subqueries

Algorithm:

1. Create two related tables.
2. Insert records into both tables.
3. Perform an inner join.
4. Execute a subquery to retrieve records satisfying a condition.

Code:

```
CREATE TABLE Dept9 (
   DeptID NUMBER PRIMARY KEY,
   DeptName VARCHAR2(30)
);

CREATE TABLE Emp9 (
   EmpID NUMBER PRIMARY KEY,
   EmpName VARCHAR2(30),
   Salary NUMBER,
   DeptID NUMBER REFERENCES Dept9(DeptID)
);

INSERT INTO Dept9 VALUES (10, 'CSE');
INSERT INTO Dept9 VALUES (20, 'ECE');

INSERT INTO Emp9 VALUES (1, 'Anu', 30000, 10);
INSERT INTO Emp9 VALUES (2, 'Ravi', 25000, 20);
INSERT INTO Emp9 VALUES (3, 'Meena', 40000, 10);

-- INNER JOIN
SELECT E.EmpName, D.DeptName
FROM Emp9 E
JOIN Dept9 D ON E.DeptID = D.DeptID;

-- Subquery
SELECT EmpName, Salary
```

```
FROM Emp9
WHERE Salary > (
   SELECT AVG(Salary) FROM Emp9
);
```

Output — Join

EmpName	DeptName
Anu	CSE
Ravi	ECE
Meena	CSE

Output — Subquery

EmpName	Salary
Meena	40000

Part B: PL/SQL Block with Conditions and User Input

Aim: To write a PL/SQL block that accepts input and checks a condition.

Algorithm:

1. Declare a variable.
2. Accept a value from the user.
3. Check whether the marks are at least 40.
4. Display the result.

Code (Oracle SQL*Plus):

```
SET SERVEROUTPUT ON;

DECLARE
   marks NUMBER := &marks;
BEGIN
   IF marks >= 40 THEN
      DBMS_OUTPUT.PUT_LINE('PASS');
   ELSE
      DBMS_OUTPUT.PUT_LINE('FAIL');
   END IF;
END;
/
```

Sample output:

```
Enter value for marks: 75
PASS
```

Part C: Exception Handling

Aim: To handle exceptions in a PL/SQL block.

Algorithm:

1. Declare two numeric variables.
2. Accept two numbers.
3. Perform division.
4. Handle division by zero and other exceptions.

Code:

```
SET SERVEROUTPUT ON;

DECLARE
   a NUMBER := &a;
   b NUMBER := &b;
   result NUMBER;
BEGIN
   result := a / b;
   DBMS_OUTPUT.PUT_LINE('Result = ' || result);
EXCEPTION
   WHEN ZERO_DIVIDE THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('Invalid value');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Other error occurred');
END;
/
```

Sample output:

```
Enter value for a: 10
Enter value for b: 0
Cannot divide by zero
```

Part D: Creating a Procedure

Aim: To create and execute a stored procedure.

Algorithm:

1. Create a procedure with an input parameter.
2. Retrieve the employee's salary.
3. Display the salary.
4. Execute the procedure using `EXEC`.

Code:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE ShowSalary(
    p_id IN NUMBER
)
AS
    v_salary NUMBER;
BEGIN
    SELECT Salary INTO v_salary
    FROM Emp9
    WHERE EmpID = p_id;

    DBMS_OUTPUT.PUT_LINE(
        'Salary = ' || v_salary
    );
EXCEPTION
```

```
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Employee not found');
END;
/

EXEC ShowSalary(1);
```

Output:

Salary = 30000

Part E: Creating a Function

Aim: To create a function that returns the square of a number.

Algorithm:

1. Create a function with one numeric parameter.
2. Calculate its square.
3. Return the result.
4. Call the function using a SQL query.

Code:

```
CREATE OR REPLACE FUNCTION SquareNum(
   n IN NUMBER
)
RETURN NUMBER
IS
BEGIN
   RETURN n * n;
END;
/

SELECT SquareNum(5) AS Result FROM DUAL;
```

Output:

RESULT

Part F: Database Triggers — INSERT, UPDATE and DELETE

Aim: To create triggers that execute automatically when insert, update, or delete operations occur.

Algorithm:

1. Create an employee table.
2. Create a log table.
3. Create an insert trigger.
4. Create update and delete triggers.
5. Perform operations and verify the log table.

Code:

```
CREATE TABLE EmpLog (
   ActionType VARCHAR2(20),
   EmpID NUMBER,
   ActionDate DATE
);

CREATE OR REPLACE TRIGGER EmpInsert
AFTER INSERT ON Emp9
FOR EACH ROW
BEGIN
   INSERT INTO EmpLog
   VALUES ('INSERT', :NEW.EmpID, SYSDATE);
END;
/

CREATE OR REPLACE TRIGGER EmpUpdate
AFTER UPDATE ON Emp9
FOR EACH ROW
BEGIN
   INSERT INTO EmpLog
   VALUES ('UPDATE', :NEW.EmpID, SYSDATE);
END;
/

CREATE OR REPLACE TRIGGER EmpDelete
```

```
AFTER DELETE ON Emp9
FOR EACH ROW
BEGIN
   INSERT INTO EmpLog
   VALUES ('DELETE', :OLD.EmpID, SYSDATE);
END;
/

INSERT INTO Emp9 VALUES (4, 'Kiran', 28000, 20);

UPDATE Emp9
SET Salary = 29000
WHERE EmpID = 4;

DELETE FROM Emp9 WHERE EmpID = 4;

SELECT ActionType, EmpID FROM EmpLog;
```

Output:

ActionType	EmpID
INSERT	4
UPDATE	4
DELETE	4

Part G: Implicit and Explicit Cursors

Aim: To demonstrate implicit and explicit cursors in PL/SQL.

Algorithm:

1. Execute a SQL statement using an implicit cursor.
2. Display the number of affected rows using `SQL%ROWCOUNT`.
3. Declare an explicit cursor.

4. Open the cursor, fetch records, and close it.

Code:

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR emp_cursor IS
      SELECT EmpID, EmpName FROM Emp9;

   v_id Emp9.EmpID%TYPE;
   v_name Emp9.EmpName%TYPE;
BEGIN
   -- Implicit cursor
   UPDATE Emp9
   SET Salary = Salary + 100
   WHERE DeptID = 10;

   DBMS_OUTPUT.PUT_LINE(
      'Rows updated: ' || SQL%ROWCOUNT
   );

   -- Explicit cursor
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO v_id, v_name;
      EXIT WHEN emp_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         v_id || ' ' || v_name
      );
   END LOOP;

   CLOSE emp_cursor;
END;
/
```

Sample output (assuming the original three employees remain):

```
Rows updated: 2
1 Anu
2 Ravi
```

3 Meena

Result: Joins, subqueries, PL/SQL blocks, exception handling, procedures, functions, triggers, and cursors were demonstrated.

SQL Plus

```
SQL> CREATE TABLE employees(
  2  employee_id NUMBER PRIMARY KEY,
  3  first_name VARCHAR2(50),
  4  salary NUMBER
  5  );

Table created.

SQL> INSERT INTO employees(employee_id,first_name,salary) VALUES(101,'sri',60000);

1 row created.

SQL> INSERT INTO employees(employee_id,first_name,salary) VALUES(102,'karthik',45000);

1 row created.

SQL> INSERT INTO employees(employee_id,first_name,salary) VALUES(103,'rama',80000);

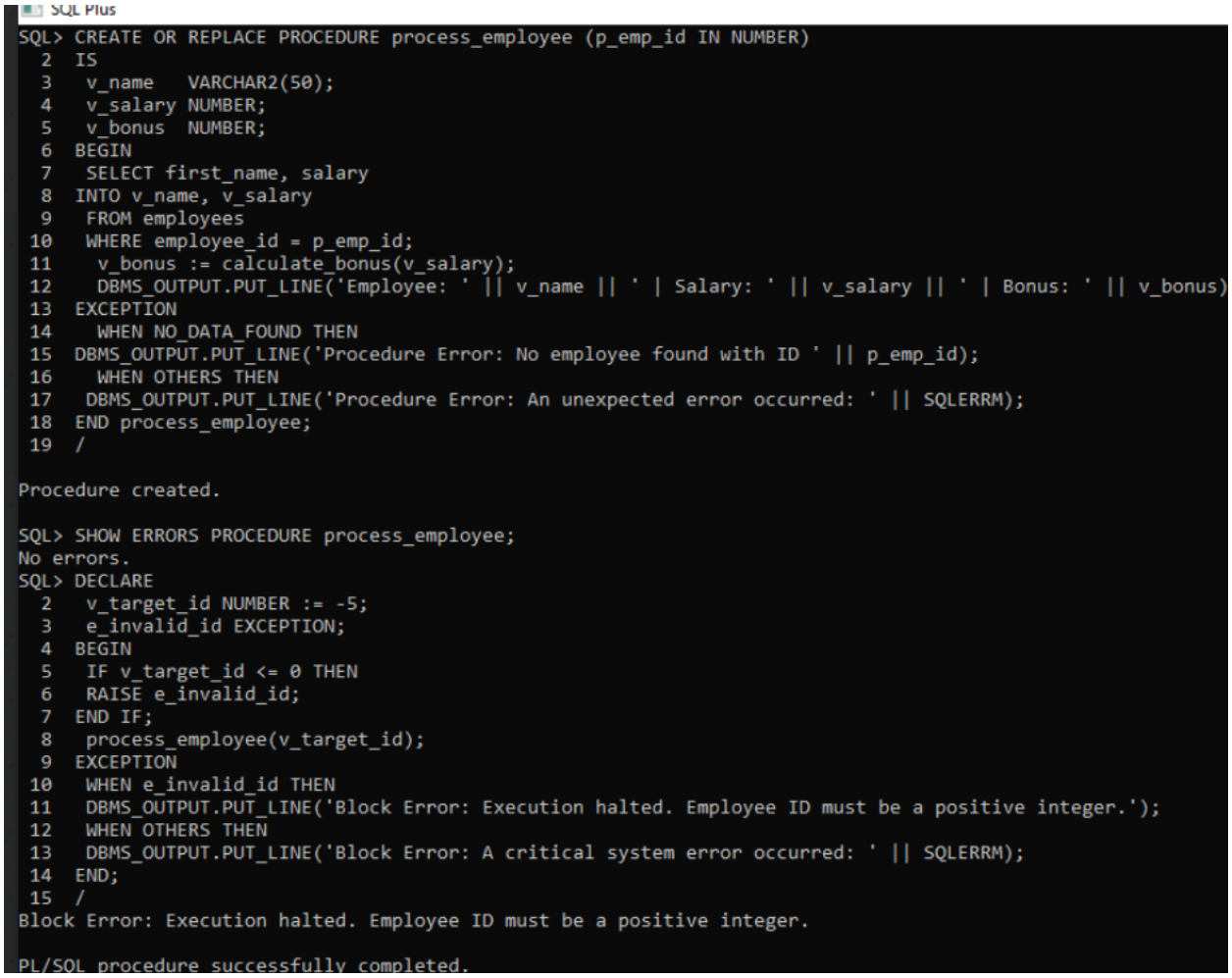
1 row created.

SQL> SELECT employee_id, first_name, salary FROM employees;

EMPLOYEE_ID FIRST_NAME                                          SALARY
----------- -------------------------------------------------- ----------
        101 sri                                                 60000
        102 karthik                                             45000
        103 rama                                                80000

SQL> SET SERVEROUTPUT ON;
SQL> CREATE OR REPLACE FUNCTION calculate_bonus (p_salary IN NUMBER)
  2  RETURN NUMBER
  3  IS
  4  v_bonus NUMBER;
  5  BEGIN
  6  v_bonus := p_salary * 0.10;
  7    RETURN v_bonus;
  8  EXCEPTION
  9   WHEN VALUE_ERROR THEN
 10  DBMS_OUTPUT.PUT_LINE('Function Error: Invalid numeric value provided.');
 11   RETURN 0;
 12  END calculate_bonus;
 13  /

Function created.
```



```
SQL Plus
SQL> CREATE OR REPLACE PROCEDURE process_employee (p_emp_id IN NUMBER)
  2  IS
  3   v_name   VARCHAR2(50);
  4   v_salary NUMBER;
  5   v_bonus  NUMBER;
  6  BEGIN
  7   SELECT first_name, salary
  8  INTO v_name, v_salary
  9   FROM employees
 10   WHERE employee_id = p_emp_id;
 11    v_bonus := calculate_bonus(v_salary);
 12    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_name || ' | Salary: ' || v_salary || ' | Bonus: ' || v_bonus)
 13  EXCEPTION
 14    WHEN NO_DATA_FOUND THEN
 15  DBMS_OUTPUT.PUT_LINE('Procedure Error: No employee found with ID ' || p_emp_id);
 16    WHEN OTHERS THEN
 17   DBMS_OUTPUT.PUT_LINE('Procedure Error: An unexpected error occurred: ' || SQLERRM);
 18  END process_employee;
 19  /

Procedure created.

SQL> SHOW ERRORS PROCEDURE process_employee;
No errors.
SQL> DECLARE
  2   v_target_id NUMBER := -5;
  3   e_invalid_id EXCEPTION;
  4  BEGIN
  5   IF v_target_id <= 0 THEN
  6   RAISE e_invalid_id;
  7  END IF;
  8   process_employee(v_target_id);
  9  EXCEPTION
 10   WHEN e_invalid_id THEN
 11   DBMS_OUTPUT.PUT_LINE('Block Error: Execution halted. Employee ID must be a positive integer.');
 12   WHEN OTHERS THEN
 13   DBMS_OUTPUT.PUT_LINE('Block Error: A critical system error occurred: ' || SQLERRM);
 14  END;
 15  /
Block Error: Execution halted. Employee ID must be a positive integer.

PL/SQL procedure successfully completed.
```

Result: Joins, subqueries, PL/SQL blocks, exception handling, procedures, functions, triggers, and cursors were demonstrated.

Experiment 10: Database Design for Management Systems

Aim: To design database tables and perform backend database operations for student, hospital, railway reservation, timetable, and hotel management systems.

General algorithm:

1. Identify the entities and their attributes.
2. Select primary keys for each table.
3. Establish relationships using foreign keys.
4. Create the tables.
5. Insert sample records.
6. Execute queries to retrieve and update information.

A. Student Information Management System

Code:

```
CREATE TABLE StudentInfo (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(30),
   Department VARCHAR(20),
   Marks INT
);

INSERT INTO StudentInfo VALUES
(101, 'Anu', 'CSE', 90),
(102, 'Ravi', 'ECE', 80);

SELECT * FROM StudentInfo;
```

Output:

StudentID	Name	Department	Marks
101	Anu	CSE	90
102	Ravi	ECE	80

B. Hospital Management System

Code:

```
CREATE TABLE Patient (
   PatientID INT PRIMARY KEY,
   PatientName VARCHAR(30),
   Disease VARCHAR(40),
```

```
   Doctor VARCHAR(30)
);

INSERT INTO Patient VALUES
(1, 'Anu', 'Fever', 'Dr. Kumar'),
(2, 'Ravi', 'Cold', 'Dr. Priya');

SELECT * FROM Patient;
```

Output:

PatientID	PatientName	Disease	Doctor
1	Anu	Fever	Dr. Kumar
2	Ravi	Cold	Dr. Priya

C. Railway Reservation System

Code:

```
CREATE TABLE Reservation (
   TicketID INT PRIMARY KEY,
   PassengerName VARCHAR(30),
   TrainNo INT,
   SeatNo INT,
   Status VARCHAR(20)
);

INSERT INTO Reservation VALUES
(1001, 'Anu', 12601, 12, 'Confirmed'),
(1002, 'Ravi', 12602, 15, 'Waiting');

SELECT * FROM Reservation;
```

Output:

TicketID	PassengerName	TrainNo	SeatNo	Status
1001	Anu	12601	12	Confirmed
1002	Ravi	12602	15	Waiting

D. Timetable Management System

Code:

```
CREATE TABLE Timetable (
   PeriodID INT PRIMARY KEY,
   DayName VARCHAR(15),
   Subject VARCHAR(30),
   StartTime VARCHAR(10),
   EndTime VARCHAR(10)
);

INSERT INTO Timetable VALUES
(1, 'Monday', 'DBMS', '09:00', '10:00'),
(2, 'Monday', 'Java', '10:00', '11:00');

SELECT * FROM Timetable;
```

Output:

PeriodID	DayName	Subject	StartTime	EndTime
1	Monday	DBMS	09:00	10:00
2	Monday	Java	10:00	11:00

E. Hotel Management System

Code:

```
CREATE TABLE HotelBooking (
   BookingID INT PRIMARY KEY,
   GuestName VARCHAR(30),
   RoomNo INT,
   Days INT,
   Amount DECIMAL(10,2)
);

INSERT INTO HotelBooking VALUES
(1, 'Anu', 101, 2, 4000),
(2, 'Ravi', 102, 3, 6000);

SELECT * FROM HotelBooking;
```

Output:

BookingID	GuestName	RoomNo	Days	Amount
1	Anu	101	2	4000.00
2	Ravi	102	3	6000.00

```
QL> CREATE TABLE Crestview_University (
  2        StudentID INT,
  3        Name VARCHAR(30),
  4        Department VARCHAR(30),
  5        Year INT
  6   );

able created.

QL> INSERT INTO Crestview_University
  2  VALUES (101, 'Arun', 'CSE', 2);

 row created.

QL>
QL> INSERT INTO Crestview_University
  2  VALUES (102, 'Priya', 'ECE', 3);

 row created.

QL>
QL> INSERT INTO Crestview_University
  2  VALUES (103, 'Rahul', 'CSE', 1);

 row created.

QL> SELECT * FROM Crestview_University;

 STUDENTID NAME                                DEPARTMENT
---------- ---------------------------------- ------------------------------
      YEAR
----------
       101 Arun                               CSE
         2

       102 Priya                              ECE
         3

       103 Rahul                              CSE
         1
```

```
SQL> SELECT Name FROM Crestview_University;

NAME
----------------------------------
Arun
Priya
Rahul
```

EXPERIMENT 3

```
SQL> CREATE TABLE Nomad_Quarters (
  2      RoomID NUMBER,
  3      GuestName VARCHAR2(30),
  4      RoomType VARCHAR2(20),
  5      Rent NUMBER
  6  );

Table created.

SQL> INSERT INTO Nomad_Quarters
  2  VALUES (101, 'Arun', 'Single', 2000);

1 row created.

SQL>
SQL> INSERT INTO Nomad_Quarters
  2  VALUES (102, 'Priya', 'Double', 3000);

1 row created.

SQL>
SQL> INSERT INTO Nomad_Quarters
  2  VALUES (103, 'Rahul', 'Single', 2200);

1 row created.

SQL> GRANT SELECT ON Nomad_Quarters TO user1;

Grant succeeded.

SQL> REVOKE SELECT ON Nomad_Quarters FROM user1;

Revoke succeeded.

SQL> COMMIT;

Commit complete.

SQL> UPDATE Nomad_Quarters
  2  SET Rent = 2500
  3  WHERE RoomID = 101;
```

```
SQL> CREATE TABLE Nomad_Quarters (
  2      RoomID NUMBER,
  3      GuestName VARCHAR2(30),
  4      RoomType VARCHAR2(20),
  5      Rent NUMBER
  6  );

Table created.

SQL> INSERT INTO Nomad_Quarters
  2  VALUES (101, 'Arun', 'Single', 2000);

1 row created.

SQL>
SQL> INSERT INTO Nomad_Quarters
  2  VALUES (102, 'Priya', 'Double', 3000);

1 row created.

SQL>
SQL> INSERT INTO Nomad_Quarters
  2  VALUES (103, 'Rahul', 'Single', 2200);

1 row created.

SQL> GRANT SELECT ON Nomad_Quarters TO user1;

Grant succeeded.

SQL> REVOKE SELECT ON Nomad_Quarters FROM user1;

Revoke succeeded.

SQL> COMMIT;

Commit complete.

SQL> UPDATE Nomad_Quarters
  2  SET Rent = 2500
  3  WHERE RoomID = 101;
```

Result: this program created and executed successfully